

S2-MAR — ANN: mide el trade-off recall / latencia

MMIA 6013 · Semana 2 · Martes

1. kNN exacto: medir cómo (no) escala
2. Mini-IVF con KMeans: recall@10 vs. speedup según `nprobe`
3. Qdrant embebido (`:memory:`) con filtros de metadata

Requisitos: `numpy`, `scikit-learn`, `matplotlib`. Qdrant: `pip install qdrant-client` (opcional — la parte 3 se salta sola si no está).

Tarea real de MMIA 6013 — actividad de clase de la Semana 2, convertida de notebook a PDF para el solver (apartado 2.a del Taller 03 v2). Se conservan el texto y el código que da el profesor; las soluciones de los ejercicios se retiraron.

Entrega: un notebook de Jupyter (`.ipynb`) **ejecutado** de principio a fin sin errores, con una celda de Markdown que encabece cada parte, los ejercicios resueltos y las reflexiones respondidas con las cifras medidas.

Código dado:

```
import time
import numpy as np
import matplotlib.pyplot as plt

rng = np.random.default_rng(7)
D = 128 # dimensión de los embeddings sintéticos

_CENTROS = np.random.default_rng(2026).normal(size=(200, D)) # "temas" fijos

def datos_realistas(n, d=D):
    """Embeddings sintéticos CON estructura: mezcla de 'temas' gaussianos fijos.
    Los embeddings reales viven agrupados por tema — crucial para que IVF
    tenga sentido (con ruido uniforme, ningún índice por clusters funciona).
    Base y consultas comparten los mismos temas, como en un corpus real."""
    asignacion = rng.integers(0, len(_CENTROS), size=n)
    X = _CENTROS[asignacion, :d] + 1.5 * rng.normal(size=(n, d))
    return X.astype(np.float32)

def normalizar(X):
    """Vectores a norma 1: así el producto punto es el coseno (sesión 06).
    Vive aquí, en el setup, porque las Partes 2 y 3 la usan aunque no hayas
    resuelto el ejercicio 1.1 (hasta el 2026-09-19 estaba dentro de la solución
    y la versión de estudiante fallaba con NameError)."""
    return X / np.linalg.norm(X, axis=-1, keepdims=True)
```

Parte 1 — kNN exacto: latencia vs. N

🔗 Ejercicio 1.1

Implementa `knn_exacto(consulta, base, k)` (vectores normalizados → usar `dot`) y mide el tiempo medio de consulta para $N = 10k, 50k, 100k, 200k$. Grafica.

```
# Ejercicio 1.1 — el costo  $O(N \cdot d)$ , medido
# (a completar)
```

Lineal en N . Con 10M de vectores y tráfico real, esto muere. Entra IVF.

Parte 2 — Mini-IVF: clustering + búsqueda local

La receta: (1) k-means agrupa la base en `nlist` celdas; (2) en consulta, comparar solo contra los centroides y buscar exacto dentro de las `nprobe` celdas más cercanas.

Código dado:

```
from sklearn.cluster import KMeans

N = 100_000
base = normalizar(datos_realistas(N))
NLIST = 64
km = KMeans(n_clusters=NLIST, n_init=3, random_state=7).fit(base)
centroides = normalizar(km.cluster_centers_.astype(np.float32))
celdas = [np.where(km.labels_ == c)[0] for c in range(NLIST)]
print(f"IVF entrenado: {NLIST} celdas, tamaño medio {np.mean([len(c) for c in
celdas]):.0f}")

# Ejercicio 2.1 – implementa knn_ivf(consulta, nprobe, k):
# 1) scores contra centroides → nprobe celdas más cercanas
# 2) kNN exacto solo dentro de esas celdas (índices globales!)
# (a completar)

# Ejercicio 2.2 – para nprobe = 1, 2, 4, 8, 16 mide:
# recall@10 (vs. knn_exacto) promedio sobre 50 consultas, y speedup vs. exacto
# (a completar)
```

La tabla del triángulo recall/latencia: con `nprobe` pequeño obtienes speedups grandes pero pierdes vecinos (los de las fronteras entre celdas); subiendo `nprobe` recuperas recall y pagas latencia. HNSW ofrece el mismo dial (`ef_search`) con mejor curva — por eso es el default de las vector DBs.

Parte 3 — Qdrant embebido: una vector DB real sin Docker

Código dado:

```
faqs = [
    ("Para resetear tu contraseña entra a Configuración > Seguridad.", "cuenta"),
    ("Los reembolsos se procesan en 5 a 7 días hábiles.", "pagos"),
    ("Puedes exportar tus datos en formato CSV desde el panel.", "cuenta"),
    ("La factura electrónica se emite al confirmar el pago.", "pagos"),
    ("El soporte atiende de lunes a viernes de 9h a 18h.", "soporte"),
    ("La API permite 100 solicitudes por minuto en el plan básico.", "api"),
]

try:
    from qdrant_client import QdrantClient
    from qdrant_client.models import Distance, VectorParams, PointStruct, Filter,
    FieldCondition, MatchValue

    import hashlib

    # embeddings sintéticos por documento (en el lab: sentence-transformers).
    # Semilla con hashlib y no hash(): hash() cambia entre procesos, y con un
    # servidor persistente los vectores deben ser iguales en cada corrida.
    def embed_demo(texto):
        semilla = int.from_bytes(hashlib.sha256(texto.encode()).digest()[:4],
"little")
        r = np.random.default_rng(semilla)
        v = r.normal(size=64).astype(np.float32)
        return (v / np.linalg.norm(v)).tolist()

    # Servidor Qdrant local (Docker en :6333); si no responde, modo embebido.
    try:
        cliente = QdrantClient(url="http://localhost:6333", timeout=3)
        cliente.get_collections()
```

```

    print("Conectado al servidor Qdrant en http://localhost:6333")
except Exception:
    cliente = QdrantClient(":memory:")
    print("Servidor no disponible — usando Qdrant embebido (:memory:)")

# el servidor persiste entre corridas: recrear la colección desde cero
if cliente.collection_exists("faqs"):
    cliente.delete_collection("faqs")
cliente.create_collection("faqs", vectors_config=VectorParams(size=64,
distance=Distance.COSINE))
cliente.upsert("faqs", points=[
    PointStruct(id=i, vector=embed_demo(t), payload={"texto": t, "tema": tema})
    for i, (t, tema) in enumerate(faqs)
])
print("Colección creada con", cliente.count("faqs").count, "puntos")

# consulta con filtro de metadata: solo tema="pagos"
hits = cliente.query_points(
    "faqs",
    query=embed_demo("¿cuándo me devuelven el dinero?"),
    query_filter=Filter(must=[FieldCondition(key="tema",
match=MatchValue(value="pagos"))]),
    limit=2,
).points
for h in hits:
    print(f"  {h.score:.3f} [{h.payload['tema']}] {h.payload['texto']}")
print("\n(Nota: con embeddings sintéticos el score no es semántico —)")
print(" el objetivo aquí es la MECÁNICA de colección/upsert/query/filtro.)")
except ImportError:
    print("qdrant-client no instalado — `pip install qdrant-client` para esta
parte.")

# Reflexión final (comentario):
# Tu proyecto tendrá ~N documentos (estímalo). Con lo medido hoy:
# a) ¿Necesitas un índice ANN o te basta kNN exacto? Justifica con números.
# b) ¿Qué motor usarías (Qdrant / pgvector / otro) y por qué operacionalmente?
#

```

Cierre

- kNN exacto: $O(N \cdot d)$, lineal — se mide, no se asume.
- IVF: carpetas + nprobe; HNSW: grafo jerárquico + ef_search. Mismo triángulo: recall / latencia / memoria.
- Qdrant: colección → upsert(vector+payload) → query con filtros.

Mañana: armamos el pipeline RAG completo — y descubrimos que el chunking importa más que el índice.